Nottingham Trent University

SCHOOL OF SCIENCE AND TECHNOLOGY

JEDUTHUN IDEMUDIA

N1044024

DISTRIBUTED DATABASE ENGINEERING LABORATORY REPORT

ANALYSIS OF CASUALTY SEVERITY IN ROAD SAFETY DATA USING HADOOP MAPREDUCE

Table of Contents

INTRODUCTION & DESIGN 2
LITERATURE REVIEW 2
TASKS & IMPLEMENTATION 3
CRITIQUE, DISCUSSION & SUMMARY 10
REFERENCES 11

INTRODUCTION & DESIGN

The increasing volume of road traffic accidents globally necessitates an in-depth analysis of road safety data to identify patterns and derive insights that can inform policy decisions and improve safety measures. This laboratory report focuses on analysing the 2018 Road Safety dataset, which contains detailed information including the index, vehicle reference, casualty reference, gender, age, severity and so on.

The main objective of this report is to employ the MapReduce programming model to efficiently process and analyse large-scale road safety data. This will be accomplished by configuring a multi-cluster and utilising the Java MapReduce model within the Hadoop framework. The MapReduce model is renowned for its capability to handle vast datasets across distributed computing environments, allowing us to dissect the data effectively.

The key components featured in the implementation of this project is the installation of Ubuntu, Java, Hadoop, Eclipse, and Spark in a multi-cluster distributed environment. After the installation is completed successfully, we will be able to run a MapReduce job to find out the total number of unique accidents occurrences within each band of casualty severity using Hadoop MapReduce framework to process the dataset efficiently. Understanding the distribution of accident severities can help to pinpoint areas requiring immediate attention and potentially guide the enhancement of road safety.

This report presents the methodology utilised to set up and execute the MapReduce job and the results obtained from the data analysis. By scrutinizing the distribution of accident frequencies across different severity levels, we aim to understand and provide valuable insights that could be used by policymakers, road safety analysts, and the public about trends and potential focal points for enhancing road safety protocols.

LITERATURE REVIEW

The analysis or road safety data through distributed computing frameworks, particularly using the MapReduce programming model, has garnered increasing attention in recent years. The MapReduce model initially introduced by Dean and Ghemawat (2008), has revolutionized the way large-scale datasets are processed, offering a scalable and efficient solution for data analysis across distributed systems.

Research by Xie et al. (2015) demonstrates the use of Hadoop-based MapReduce for traffic incident analysis, underscoring the model's effectiveness in processing vast datasets to extract meaningful insights. Similarly, Kumar and Reddy (2017) explored the application of Hadoop in analysing road accident datasets, revealing patterns that could assist in proactive safety planning.

The integration of Apache Spark alongside Hadoop, as indicated in our methodology, resonates with findings from Zaharia et al. (2016), who illustrated Spark's enhanced performance in iterative data processing tasks common in traffic data analysis. This combination harnesses Hadoop's robust storage capabilities and Spark's advanced analytics prowess, offering a comprehensive big data solution.

In the realm of road safety, studies like that of Chen et al. (2019) utilized big data analytics to identify high-risk road segments, demonstrating how data-driven insights can guide infrastructure improvements and policy reforms. The current project's focus on casualty severity analysis contributes to this body of work, aiming to provide a granular understanding of accident dynamics.

Road traffic accidents pose a significant global challenge, as highlighted by the World Health Organization (WHO), which emphasizes the necessity of robust data analysis to inform effective road safety strategies. The MapReduce model's application in this context aligns with the growing trend of leveraging big data analytics to enhance public safety measures.

In conclusion, the literature reinforces the significance of employing advanced data processing frameworks like MapReduce within Hadoop to address critical societal issues like road safety. This project's methodology aligns with established research, furthering the application of big data analytics in public health and safety domains.

TASKS & IMPLEMENTATION

Setting up a multi node pseudo distributed Hadoop cluster was the first step towards using the Hadoop MapReduce framework. The pre-requisite for setting up the multi node cluster include the installation of virtualisation software (VMware/Virtual Box), installation of Ubuntu, installation of Hadoop and Java. Ubuntu is a Linux distribution open-source software and the operating system used for each virtual machine in the cluster. Utility tools are installed in addition to the operating system. Running commands like "sudo apt-get install net-tools", "sudo apt-get install ssh", "sudo apt-get install ssh" help to achieve utility tools installation. The Java download file is then untarred, and the environment variables in the bashrc file are updated both in root & defined user. To check and confirm if java is installed, I ran the command "java -version".

```
n1044024@machine01:~$ java -version
java version "1.8.0_251"
Java(TM) SE Runtime Environment (build 1.8.0_251-b08)
Java HotSpot(TM) 64-Bit Server VM (build 25.251-b08, mixed mode)
```

Running the command "sudo-apt-get update" helps the system to update the package index and upgrade the installed packages to the newest available versions. The Hadoop download file is also untarred, and the environment variables are updated too. The configuration files core-site.xml, mapred-site.xml, hdfs-site.xml, yarn-site.xml, slaves and files which can be found in this directory "/usr/local/Hadoop/etc/Hadoop" are changed. The path in Hadoop-env.sh and /etc/environment are updated to include the Hadoop binary directories.

n1044024@machine01: ~

```
GNU nano 6.2                    .bashrc
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
  if [ -f /usr/share/bash-completion/bash_completion ]; then
    . /usr/share/bash-completion/bash_completion
  elif [ -f /etc/bash_completion ]; then
    . /etc/bash_completion
  fi
fi

export JAVA_HOME=/usr/lib/jvm/jdk1.8.0_251
export PATH=$PATH:$JAVA_HOME/bin
export HADOOP_HOME=/usr/local/hadoop
export HADOOP_INSTALL=/usr/local/hadoop
export PATH=$PATH:$HADOOP_INSTALL/bin
export PATH=$PATH:$HADOOP_INSTALL/sbin
export HADOOP_CONF_DIR=$HADOOP_INSTALL/etc/hadoop
export SPARK_HOME=/usr/local/spark
export PATH=$PATH:$SPARK_HOME/bin
export PATH=$PATH:$SPARK_HOME/sbin
```

n1044024@machine01: /usr/local/hadoop/etc/hadoop

```
GNU nano 6.2                                        core-site.xml
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<!--
  Licensed under the Apache License, Version 2.0 (the "License");
  you may not use this file except in compliance with the License.
  You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

  Unless required by applicable law or agreed to in writing, software
  distributed under the License is distributed on an "AS IS" BASIS,
  WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
  See the License for the specific language governing permissions and
  limitations under the License. See accompanying LICENSE file.
-->

<!-- Put site-specific property overrides in this file. -->

<configuration>
<property>
<name>fs.defaultFS</name>
<value>hdfs://machine01:9000</value>
</property>
</configuration>
```

n1044024@machine01: /usr/local/hadoop/etc/hadoop

```
GNU nano 6.2                                                    mapred-site.xml
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<!--
  Licensed under the Apache License, Version 2.0 (the "License");
  you may not use this file except in compliance with the License.
  You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

  Unless required by applicable law or agreed to in writing, software
  distributed under the License is distributed on an "AS IS" BASIS,
  WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
  See the License for the specific language governing permissions and
  limitations under the License. See accompanying LICENSE file.
-->

<!-- Put site-specific property overrides in this file. -->

<configuration>
<property>
<name>mapreduce.framework.name</name>
<value>yarn</value>
</property>
<property>
<name>yarn.app.mapreduce.am.env</name>
<value>HADOOP_MAPRED_HOME=/usr/local/hadoop</value>
</property>
<property>
<name>mapreduce.map.env</name>
<value>HADOOP_MAPRED_HOME=/usr/local/hadoop</value>
</property>
<property>
<name>mapreduce.reduce.env</name>
<value>HADOOP_MAPRED_HOME=/usr/local/hadoop</value>
</property>
</configuration>
```

n1044024@machine01: /usr/local/hadoop/etc/hadoop

```
GNU nano 6.2                                                    hdfs-site.xml
  You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

  Unless required by applicable law or agreed to in writing, software
  distributed under the License is distributed on an "AS IS" BASIS,
  WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
  See the License for the specific language governing permissions and
  limitations under the License. See accompanying LICENSE file.
-->

<!-- Put site-specific property overrides in this file. -->

<configuration>
<property>
<name>dfs.webhdfs.enabled</name>
<value>true</value>
</property>
<property>
<name>dfs.replication</name>
<value>1</value>
</property>
<property>
<name>dfs.namenode.name.dir</name>
<value>/abc/name</value>
<final>true</final>
</property>
<property>
<name>dfs.datanode.data.dir</name>
<value>/abc/data1</value>
<final>true</final>
</property>
<property>
<name>dfs.namenode.http-address</name>
<value>machine01:50070</value>
</property>
<property>
<name>dfs.namenode.secondary.http-address</name>
<value>machine02:50090</value>
</property>
</configuration>
```

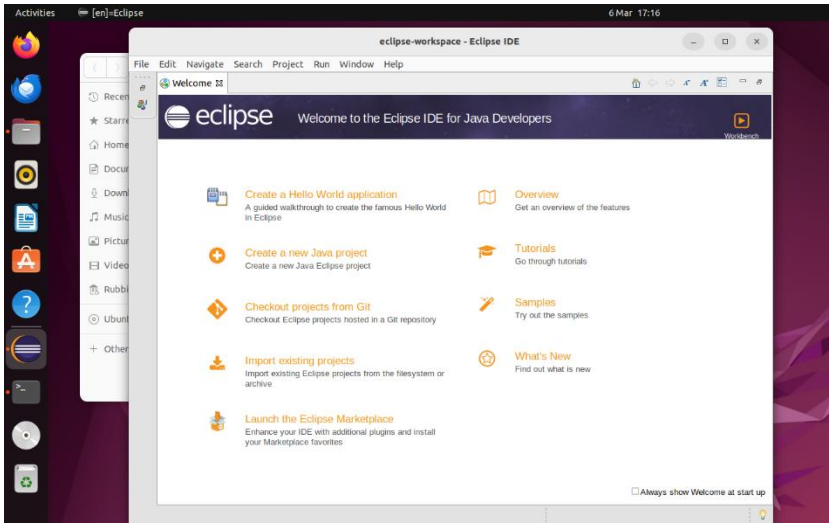
```
n1044024@machine01: /usr/local/hadoop/etc/hadoop
  GNU nano 6.2                                                    slaves
machine01
machine02
machine03
machine04
machine05
```

```
n1044024@machine01: /usr/local/hadoop/etc/hadoop
  GNU nano 6.2                                                    yarn-site.xml
  Unless required by applicable law or agreed to in writing, software
  distributed under the License is distributed on an "AS IS" BASIS,
  WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
  See the License for the specific language governing permissions and
  limitations under the License. See accompanying LICENSE file.
-->
<configuration>
<property>
<name>yarn.resourcemanager.address</name>
<value>machine01:9001</value>
</property>
<property>
<name>yarn.resourcemanager.resource-tracker.address</name>
<value>machine01:8031</value>
</property>
<property>
<name>yarn.nodemanager.aux-services.mapreduce_shuffle.class</name>
<value>org.apache.hadoop.mapred.ShuffleHandler</value>
</property>
<property>
<name>yarn.nodemanager.aux-services</name>
<value>mapreduce_shuffle</value>
</property>
<property>
<name>yarn.nodemanager.pmem-check-enabled</name>
<value>false</value>
</property>
<property>
<name>yarn.nodemanager.vmem-check-enabled</name>
<value>false</value>
</property>
<property>
<name>yarn.log-aggregation-enable</name>
<value>true</value>
</property>
<property>
<name>yarn.nodemanager.log-aggregation-roll-monitoring-interval-seconds</name>
<value>3600</value>
</property>
</configuration>
```

To check and confirm if Hadoop is installed, I ran the command “hadoop version”.

```
n1044024@machine01:~$ hadoop version
Hadoop 2.9.0
Subversion https://git-wip-us.apache.org/repos/asf/hadoop.git -r 756ebc8394e473ac25feac05fa493f6d612e6c50
Compiled by arsuresh on 2017-11-13T23:15Z
Compiled with protoc 2.5.0
From source with checksum 0a76a9a32a5257331741f8d5932f183
This command was run using /usr/local/hadoop-2.9.0/share/hadoop/common/hadoop-common-2.9.0.jar
```

The configuration files in all the virtual machines differ and is configured based on their roles. Machine01 being the name node is responsible for metadata storage, cluster management, block management, data node management, client interaction and so on. Machine02 being the secondary name node is responsible for checkpointing. Machine03 – 05 are the data nodes which together with the name node and secondary name node perform the MapReduce job. A password less connection in-between all machines is made, and the /etc/hosts configuration file is changed to include the Ip address and hostname of each virtual machine.



After Java, Hadoop and eclipse is installed, the input data dftRoadSafetyData_Casualties_2018.csv file is downloaded. A Mapper, Reducer and Driver Java class files are created to handle the MapReduce job. The Java class paths are included in the bashrc file. The command “export CLASSPATH = “$HADOOP_HOME/share/hadoop/mapreduce/*:$HADOOP_HOME/share/hadoop/common/*:~/MapReduceRoadSafetyData/roadsafetydata/*:$HADOOP_HOME/lib/*" contains the directory of the java files and the package “roadsafetydata”. The Hadoop cluster is started, and the input file is put into the cluster directory using the command “hadoop fs -put /home/n1044024/MapReduceRoadSafetyData/ dftRoadSafetyData_Casualties_2018.csv /user/n1044024/JavaExampleDFT2”. To perform the MapReduce job the command “hadoop jar AccidentIndexPerCasualtySeverity.jar /user/n104024/JavaExampleDFT2 /user/n1044024/JavaExampleDFT2/DFT_out”. I was also able to install Apache spark.

```
n1044024@machine01:~/MapReduceRoadSafetyData$ start-all.sh
This script is Deprecated. Instead use start-dfs.sh and start-yarn.sh
Starting namenodes on [machine01]
machine01: starting namenode, logging to /usr/local/hadoop-2.9.0/logs/hadoop-n1044024-namenode-machine01.out
machine05: starting datanode, logging to /usr/local/hadoop-2.9.0/logs/hadoop-n1044024-datanode-machine05.out
machine04: starting datanode, logging to /usr/local/hadoop-2.9.0/logs/hadoop-n1044024-datanode-machine04.out
machine01: starting datanode, logging to /usr/local/hadoop-2.9.0/logs/hadoop-n1044024-datanode-machine01.out
machine03: starting datanode, logging to /usr/local/hadoop-2.9.0/logs/hadoop-n1044024-datanode-machine03.out
machine02: starting datanode, logging to /usr/local/hadoop-2.9.0/logs/hadoop-n1044024-datanode-machine02.out
Starting secondary namenodes [machine02]
machine02: starting secondarynamenode, logging to /usr/local/hadoop-2.9.0/logs/hadoop-n1044024-secondarynamenode-machine02.out
starting yarn daemons
starting resourcemanager, logging to /usr/local/hadoop-2.9.0/logs/yarn-n1044024-resourcemanager-machine01.out
machine02: starting nodemanager, logging to /usr/local/hadoop-2.9.0/logs/yarn-n1044024-nodemanager-machine02.out
machine05: starting nodemanager, logging to /usr/local/hadoop-2.9.0/logs/yarn-n1044024-nodemanager-machine05.out
machine03: starting nodemanager, logging to /usr/local/hadoop-2.9.0/logs/yarn-n1044024-nodemanager-machine03.out
machine04: starting nodemanager, logging to /usr/local/hadoop-2.9.0/logs/yarn-n1044024-nodemanager-machine04.out
machine01: starting nodemanager, logging to /usr/local/hadoop-2.9.0/logs/yarn-n1044024-nodemanager-machine01.out
n1044024@machine01:~/MapReduceRoadSafetyData$ hadoop fs -ls /user/n1044024/JavaExampleDFT2
Found 2 items
drwxr-xr-x   - n1044024 supergroup          0 2024-03-11 18:50 /user/n1044024/JavaExampleDFT2/DFT_out
-rw-r--r--   1 n1044024 supergroup    7466192 2024-03-11 16:15 /user/n1044024/JavaExampleDFT2/dftRoadSafetyData_Casualties_2018.csv
n1044024@machine01:~/MapReduceRoadSafetyData$ hadoop fs -ls /user/n1044024/JavaExampleDFT2/DFT_out
Found 2 items
-rw-r--r--   1 n1044024 supergroup          0 2024-03-11 18:50 /user/n1044024/JavaExampleDFT2/DFT_out/_SUCCESS
-rw-r--r--   1 n1044024 supergroup         44 2024-03-11 18:50 /user/n1044024/JavaExampleDFT2/DFT_out/part-00000
n1044024@machine01:~/MapReduceRoadSafetyData$ hadoop fs -cat /user/n1044024/JavaExampleDFT2/DFT_out/part-00000
1       1671
2       23500
3       102705
Casualty_Severity       1
n1044024@machine01:~/MapReduceRoadSafetyData$
```

```
n1044024@machine01: ~/MapReduceNHS
n1044024@machine01:~/MapReduceNHS$  hadoop jar SickRatePerOrgCode.jar /user/n1044024/JavaExampleNHS2 /user/n1044024/JavaExampleNHS2/NHS_out
24/03/06 12:51:43 INFO client.RMProxy: Connecting to ResourceManager at machine01/192.168.241.129:9001
24/03/06 12:51:44 INFO client.RMProxy: Connecting to ResourceManager at machine01/192.168.241.129:9001
24/03/06 12:51:45 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
24/03/06 12:51:46 INFO mapred.FileInputFormat: Total input files to process : 1
24/03/06 12:51:46 INFO mapreduce.JobSubmitter: number of splits:2
24/03/06 12:51:46 INFO Configuration.deprecation: yarn.resourcemanager.system-metrics-publisher.enabled is deprecated. Instead, use yarn.system-metrics-publisher.enabled
24/03/06 12:51:47 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1709729396728_0001
24/03/06 12:51:48 INFO impl.YarnClientImpl: Submitted application application_1709729396728_0001
24/03/06 12:51:48 INFO mapreduce.Job: The url to track the job: http://machine01:8088/proxy/application_1709729396728_0001/
24/03/06 12:51:48 INFO mapreduce.Job: Running job: job_1709729396728_0001

24/03/06 12:52:04 INFO mapreduce.Job: Job job_1709729396728_0001 running in uber mode : false
24/03/06 12:52:04 INFO mapreduce.Job:  map 0% reduce 0%
24/03/06 12:52:26 INFO mapreduce.Job:  map 100% reduce 0%
24/03/06 12:52:36 INFO mapreduce.Job:  map 100% reduce 100%
24/03/06 12:52:37 INFO mapreduce.Job: Job job_1709729396728_0001 completed successfully
24/03/06 12:52:38 INFO mapreduce.Job: Counters: 49
        File System Counters
                FILE: Number of bytes read=524751
                FILE: Number of bytes written=1656424
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read=6179372
                HDFS: Number of bytes written=5665
                HDFS: Number of read operations=9
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=2
        Job Counters
                Launched map tasks=2
                Launched reduce tasks=1
                Data-local map tasks=2
                Total time spent by all maps in occupied slots (ms)=40143
                Total time spent by all reduces in occupied slots (ms)=7938
                Total time spent by all map tasks (ms)=40143
                Total time spent by all reduce tasks (ms)=7938
                Total vcore-milliseconds taken by all map tasks=40143
                Total vcore-milliseconds taken by all reduce tasks=7938
                Total megabyte-milliseconds taken by all map tasks=41106432
                Total megabyte-milliseconds taken by all reduce tasks=8128512
        Map-Reduce Framework
                Map input records=40315
                Map output records=40315
                Map output bytes=444115
                Map output materialized bytes=524757
                Input split bytes=298
                Combine input records=0
                Combine output records=0
                Reduce input groups=625
                Reduce shuffle bytes=524757
                Reduce input records=40315
                Reduce output records=625
                Spilled Records=80630
                Shuffled Maps =2
                Failed Shuffles=0
```

```
n1044024@machine01: /usr/local/spark/bin
machine02: full log in /usr/local/saprk/conf/spark-n1044024-org.apache.spark.deploy.worker.Worker-1-machine02.out
n1044024@machine01:/usr/local/spark/sbin$ ./stop-all.sh
machine04: stopping org.apache.spark.deploy.worker.Worker
machine02: no org.apache.spark.deploy.worker.Worker to stop
machine05: stopping org.apache.spark.deploy.worker.Worker
localhost: stopping org.apache.spark.deploy.worker.Worker
machine03: stopping org.apache.spark.deploy.worker.Worker
machine01: no org.apache.spark.deploy.worker.Worker to stop
stopping org.apache.spark.deploy.master.Master
n1044024@machine01:/usr/local/spark/sbin$ ./start-all.sh
starting org.apache.spark.deploy.master.Master, logging to /usr/local/spark/conf/spark-n1044024-org.apache.spark.deploy.master.Master-1-machine01.out
machine05: starting org.apache.spark.deploy.worker.Worker, logging to /usr/local/spark/conf/spark-n1044024-org.apache.spark.deploy.worker.Worker-1-machine05.out
machine03: starting org.apache.spark.deploy.worker.Worker, logging to /usr/local/spark/conf/spark-n1044024-org.apache.spark.deploy.worker.Worker-1-machine03.out
machine04: starting org.apache.spark.deploy.worker.Worker, logging to /usr/local/spark/conf/spark-n1044024-org.apache.spark.deploy.worker.Worker-1-machine04.out
localhost: starting org.apache.spark.deploy.worker.Worker, logging to /usr/local/spark/conf/spark-n1044024-org.apache.spark.deploy.worker.Worker-1-machine01.out
machine02: starting org.apache.spark.deploy.worker.Worker, logging to /usr/local/spark/conf/spark-n1044024-org.apache.spark.deploy.worker.Worker-1-machine02.out
machine01: starting org.apache.spark.deploy.worker.Worker, logging to /usr/local/spark/conf/spark-n1044024-org.apache.spark.deploy.worker.Worker-1-machine01.out
n1044024@machine01:/usr/local/spark/sbin$ ./stop-all.sh
machine04: stopping org.apache.spark.deploy.worker.Worker
machine05: stopping org.apache.spark.deploy.worker.Worker
machine03: stopping org.apache.spark.deploy.worker.Worker
machine02: stopping org.apache.spark.deploy.worker.Worker
localhost: stopping org.apache.spark.deploy.worker.Worker
machine01: stopping org.apache.spark.deploy.worker.Worker
stopping org.apache.spark.deploy.master.Master
n1044024@machine01:/usr/local/spark/sbin$ ./start-all.sh
starting org.apache.spark.deploy.master.Master, logging to /usr/local/spark/conf/spark-n1044024-org.apache.spark.deploy.master.Master-1-machine01.out
machine04: starting org.apache.spark.deploy.worker.Worker, logging to /usr/local/spark/conf/spark-n1044024-org.apache.spark.deploy.worker.Worker-1-machine04.out
machine05: starting org.apache.spark.deploy.worker.Worker, logging to /usr/local/spark/conf/spark-n1044024-org.apache.spark.deploy.worker.Worker-1-machine05.out
machine03: starting org.apache.spark.deploy.worker.Worker, logging to /usr/local/spark/conf/spark-n1044024-org.apache.spark.deploy.worker.Worker-1-machine03.out
machine02: starting org.apache.spark.deploy.worker.Worker, logging to /usr/local/spark/conf/spark-n1044024-org.apache.spark.deploy.worker.Worker-1-machine02.out
localhost: starting org.apache.spark.deploy.worker.Worker, logging to /usr/local/spark/conf/spark-n1044024-org.apache.spark.deploy.worker.Worker-1-machine01.out
machine01: starting org.apache.spark.deploy.worker.Worker, logging to /usr/local/spark/conf/spark-n1044024-org.apache.spark.deploy.worker.Worker-1-machine01.out
n1044024@machine01:/usr/local/spark/sbin$ ./spark-shell
bash: ./spark-shell: No such file or directory
n1044024@machine01:/usr/local/spark/sbin$ cd /usr/local/spark/bin
n1044024@machine01:/usr/local/spark/bin$ ./spark-shell
24/03/06 17:09:35 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Spark context Web UI available at http://machine01:4040
Spark context available as 'sc' (master = local[*], app id = local-1709744981574).
Spark session available as 'spark'.
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.0.0
      /_/

Using Scala version 2.12.10 (Java HotSpot(TM) 64-Bit Server VM, Java 1.8.0_251)
Type in expressions to have them evaluated.
Type :help for more information.

scala>
```

```
n1044024@machine01: ~/MapReduceNHS
                FILE: Number of bytes written=1656424
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read=6179372
                HDFS: Number of bytes written=5665
                HDFS: Number of read operations=9
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=2
        Job Counters
                Launched map tasks=2
                Launched reduce tasks=1
                Data-local map tasks=2
                Total time spent by all maps in occupied slots (ms)=40143
                Total time spent by all reduces in occupied slots (ms)=7938
                Total time spent by all map tasks (ms)=40143
                Total time spent by all reduce tasks (ms)=7938
                Total vcore-milliseconds taken by all map tasks=40143
                Total vcore-milliseconds taken by all reduce tasks=7938
                Total megabyte-milliseconds taken by all map tasks=41106432
                Total megabyte-milliseconds taken by all reduce tasks=8128512
        Map-Reduce Framework
                Map input records=40315
                Map output records=40315
                Map output bytes=444115
                Map output materialized bytes=524757
                Input split bytes=298
                Combine input records=0
                Combine output records=0
                Reduce input groups=625
                Reduce shuffle bytes=524757
                Reduce input records=40315
                Reduce output records=625
                Spilled Records=80630
                Shuffled Maps =2
                Failed Shuffles=0
                Merged Map outputs=2
                GC time elapsed (ms)=1171
                CPU time spent (ms)=5840
                Physical memory (bytes) snapshot=790593536
                Virtual memory (bytes) snapshot=5905031168
                Total committed heap usage (bytes)=523763712
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=6179074
        File Output Format Counters
                Bytes Written=5665
n1044024@machine01:~/MapReduceNHS$
```

Block information -- Block 0

Block ID: 1073741945

Block Pool ID: BP-1331639815-192.168.241.129-1709050964778

Generation Stamp: 1121

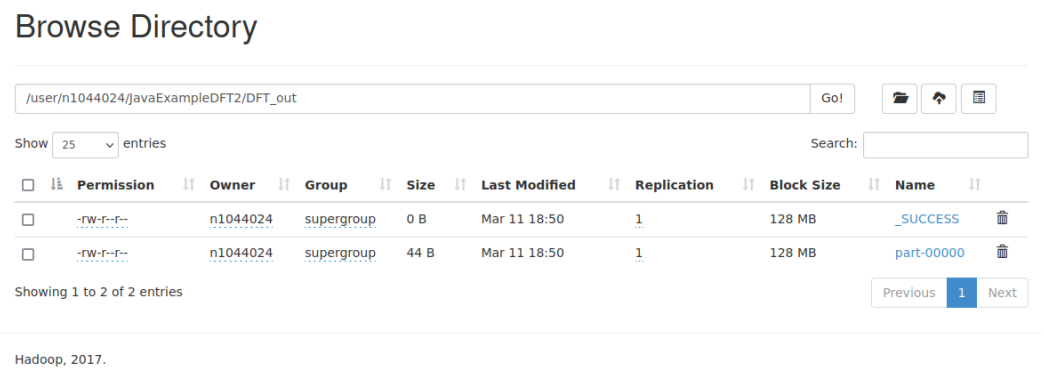
Size: 44

Availability:

- machine04

File contents

```
1       1671
2       23500
3       102705
Casualty_Severity   1
```



CRITIQUE, DISCUSSION & SUMMARY

The MapReduce job was tasked with analysing the 2018 Road Safety dataset, focusing on counting the unique occurrences of accident indices within each band of casualty severity. The results were as follows:

- Severity Band 1: 1,671 unique accidents
- Severity Band 2: 23,500 unique accidents
- Severity Band 3: 102,705 unique accidents

The job effectively categorized accidents into different severity bands, revealing a progressive increase in accident counts from Band 1 to Band 3. This distribution suggests that accidents in Band 3, which typically represent less severe incidents, are far more common than those in Bands 1 and 2. This insight is crucial for data-driven decision-making in road safety management. However, the analysis could be deepened further by segmenting the data based on additional parameters like location and time, resulting in a more detailed understanding of accident patterns. While the job efficiently utilized Hadoop's MapReduce for data processing, there is room for optimization in terms of resource allocation and job tuning to enhance performance. Additionally, the integration of real-time data processing and advanced analytics, such as machine learning, could offer predictive insights, shifting the analysis from descriptive to prescriptive. Ensuring high availability of the Hadoop cluster and facilitating interactive data exploration through tools like Hive or Drill could also significantly increase the usability and accessibility of the analysis, making the insights more actionable for a larger range of stakeholders.

In summary, while the MapReduce job successfully delivered the intended results, there is significant potential to expand the scope and depth of the analysis. By incorporating additional data dimensions, enhancing real-time processing capabilities, and integrating predictive analytics, the findings could become even more impactful, providing a robust foundation for informed policy-making and strategic initiatives in road safety.

REFERENCES

Databricks. (2021). *What is MapReduce?* [online] Available at: https://www.databricks.com/glossary/mapreduce#:~:text=MapReduce%20was%20developed%20in%20the,commonly%20used%20in%20functional%20programming. [Accessed 12 Mar. 2024].

Ruchitha Geebu (2021). *History and Advantages of Hadoop MapReduce Programming*. [online] Mindmajix. Available at: https://mindmajix.com/mapreduce/history-and-advantages-of-hadoop-mapreduce-programming [Accessed 12 Mar. 2024].

Tayar, Y., Satyanarayana, S. and Sivaram Prasad, R. (n.d.). A SURVEY NOTE ON BIG DATA ANALYTICS FRAMEWORKS ON THE HADOOP PLATFORM Section A-Research paper A Survey Note on Big Data Analytics Frameworks on the Hadoop Platform. *Eur. Chem. Bull*, [online] 2023, pp.1336–1348. doi:https://doi.org/10.48047/ecb/2023.12.si12.121.

Ma, C., Zhao, M. and Zhao, Y. (2023). An overview of Hadoop applications in transportation big data. *Journal of Traffic and Transportation Engineering (English Edition)*, [online] 10(5), pp.900–917. doi:https://doi.org/10.1016/j.jtte.2023.05.003.

Salloum, S., Ruslan Dautov, Chen, X., Patrick Xiaogang Peng and Joshua Zhexue Huang (2016). Big data analytics on Apache Spark. *International Journal of Data Science and Analytics*, [online] 1(3-4), pp.145–164. doi:https://doi.org/10.1007/s41060-016-0027-9.

Comi, A., Polimeni, A. and Balsamo, C. (2022). Road Accident Analysis with Data Mining Approach: evidence from Rome. *Transportation Research Procedia*, [online] 62, pp.798–805. doi:https://doi.org/10.1016/j.trpro.2022.02.099.